

WEEK8-Homework-Ping CHEN

Test website: <https://bitetoothpastebits.com/>

1. Bug report

Bug ID: BUG-001

Title: List items are not announced with sequence information by screen reader (NVDA).

Description: The screen reader fails to announce the sequence of how to use the product, making it difficult for users to understand the order of steps.

Steps to Reproduce:

1. Navigate to <https://bitetoothpastebits.com/>
2. Use a screen reader (NVDA)
3. Navigate "How do I use Toothpaste Bits?"
4. Navigate the answer to the question.

Expected Result: The screen reader can announce the sequence and the steps.

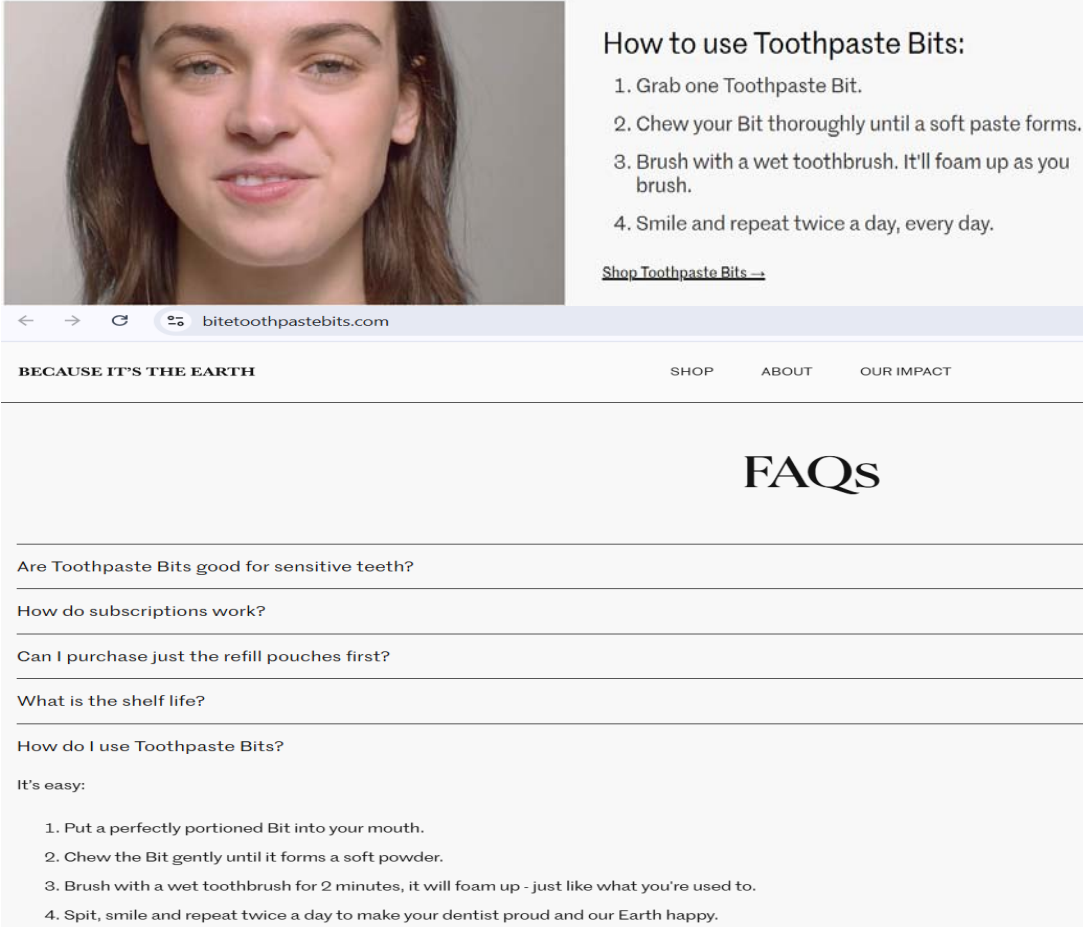
Actual Result: The screen reader fails to announce the sequence of how to use the product.

Severity: major.

Priority: high.

Environment: Chrome

Attachments:



How to use Toothpaste Bits:

1. Grab one Toothpaste Bit.
2. Chew your Bit thoroughly until a soft paste forms.
3. Brush with a wet toothbrush. It'll foam up as you brush.
4. Smile and repeat twice a day, every day.

[Shop Toothpaste Bits ->](#)

BECAUSE IT'S THE EARTH **SHOP** **ABOUT** **OUR IMPACT**

FAQs

Are Toothpaste Bits good for sensitive teeth?

How do subscriptions work?

Can I purchase just the refill pouches first?

What is the shelf life?

How do I use Toothpaste Bits?

It's easy:

1. Put a perfectly portioned Bit into your mouth.
2. Chew the Bit gently until it forms a soft powder.
3. Brush with a wet toothbrush for 2 minutes, it will foam up - just like what you're used to.
4. Spit, smile and repeat twice a day to make your dentist proud and our Earth happy.

Reported By: Ping CHEN

Date:07/04/2026

Bug ID: BUG-002

Title: Images for reviews are not announced by screen reader (NVDA).

Description: The review image elements are lacking alt attributes. As a result, screen readers cannot convey the content of these images to users.

Steps to Reproduce:

1. Navigate to <https://bitetoothpastebits.com/>
2. Enable a screen reader (NVDA)
3. Navigate “21,000+Reviews for a Reason”
4. Navigate the images of reviews.

Expected Result: The screen reader can announce the content of the images.

Actual Result: The screen reader fails to announce the content of the images.

Severity: minor.

Priority: low.

Environment: Chrome

Attachments:

21,000+ Reviews for
a Reason

Don't just take our word for it.

SHOP BEST SELLERS



Reported By: Ping CHEN

Date:07/04/2026

Bug ID: BUG-003

Title: Rating stars are not announced by screen readers (NVDA).

Description: The rating stars for the reviews are not clearly exposed to users. Screen readers cannot announce the rating value.

Steps to Reproduce:

1. Navigate to <https://bitetoothpastebits.com/>
2. Enable a screen reader (NVDA)
3. Navigate “21,000+Reviews for a Reason”
4. Navigate the rating section of the reviews.

Expected Result: The screen reader can clearly announce the rating value.

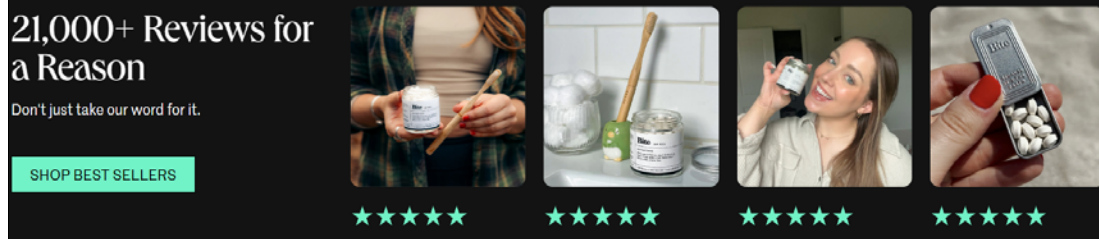
Actual Result: No rating information is announced.

Severity: major.

Priority: high.

Environment: Chrome

Attachments:



Reported By: Ping CHEN

Date:07/04/2026

Bug ID: BUG-004

Title: Collapsed shop accordion content is announced by screen reader (NVDA).

Description: The accordion content inside the "Shop" section is announced by screen reader even when the accordion is collapsed. This exposes hidden content to screen reader users and creates inconsistency and confusion between visual and screen reader information.

Steps to Reproduce:

1. Navigate to <https://bitetoothpastebits.com/>
2. Enable a screen reader (NVDA)
3. Navigate "shop"
4. Continue to navigate with "Tab".

Expected Result: Hidden content should not be read or focusable when the accordion is collapsed.

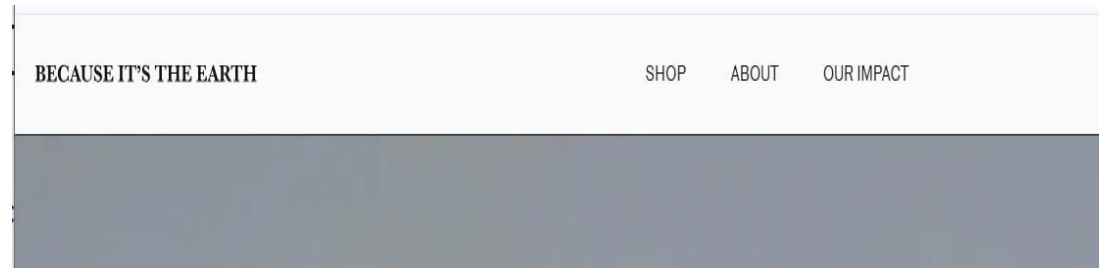
Actual Result: Screen reader announces content inside the collapsed accordion.

Severity: major.

Priority: high.

Environment: Chrome

Attachments:



Reported By: Ping CHEN

Date:07/04/2026

Bug ID: BUG-005

Title: Insufficient color contrast between text and background on Chrome.

Description: The white text displayed on the page has low contrast against its grey background, making it difficult to read, especially for users with visual impairments.

Steps to Reproduce:

1. Navigate to <https://bitetoothpastebits.com/>
2. Observe the grey image background and the text “22,000+Five-Star Reviews Get plastic out of your routine. Science-backed. Planet-positive. Made with clean ingredients. It’s time for your routine upgrade.”
3. Use a contrast checker tool to check the color.

Expected Result: Text should have sufficient contrast against the background.

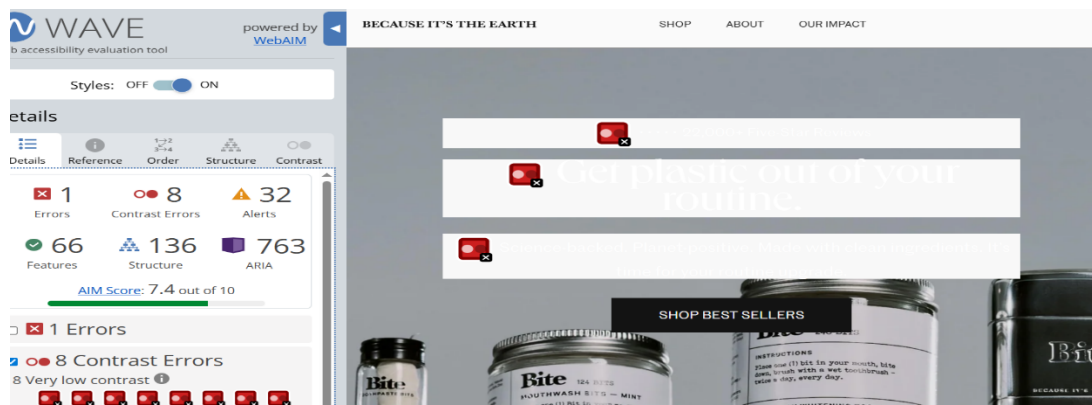
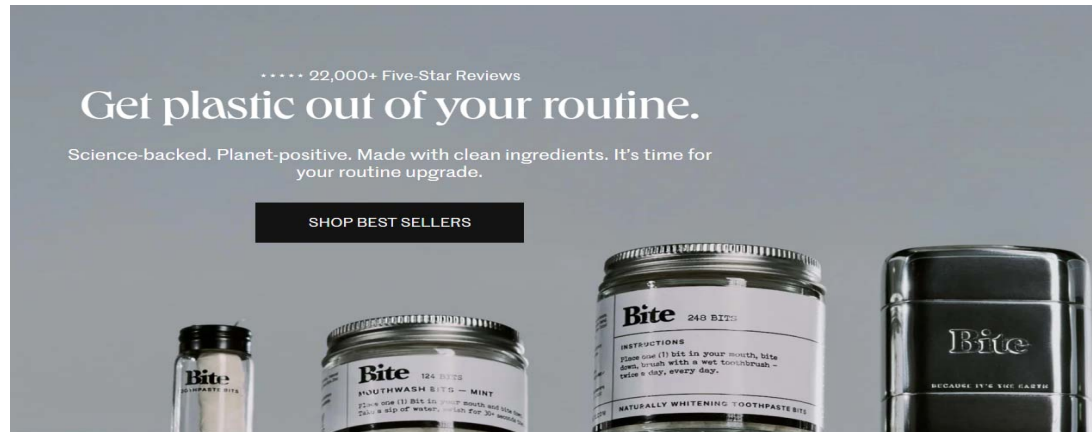
Actual Result: Contrast ratio is 1.06:1, which is below what WCAG recommended. According to WCAG 2.1, Normal text: at least 4.5:1; Large text: at least 3:1.

Severity: major.

Priority: high.

Environment: Chrome

Attachments:

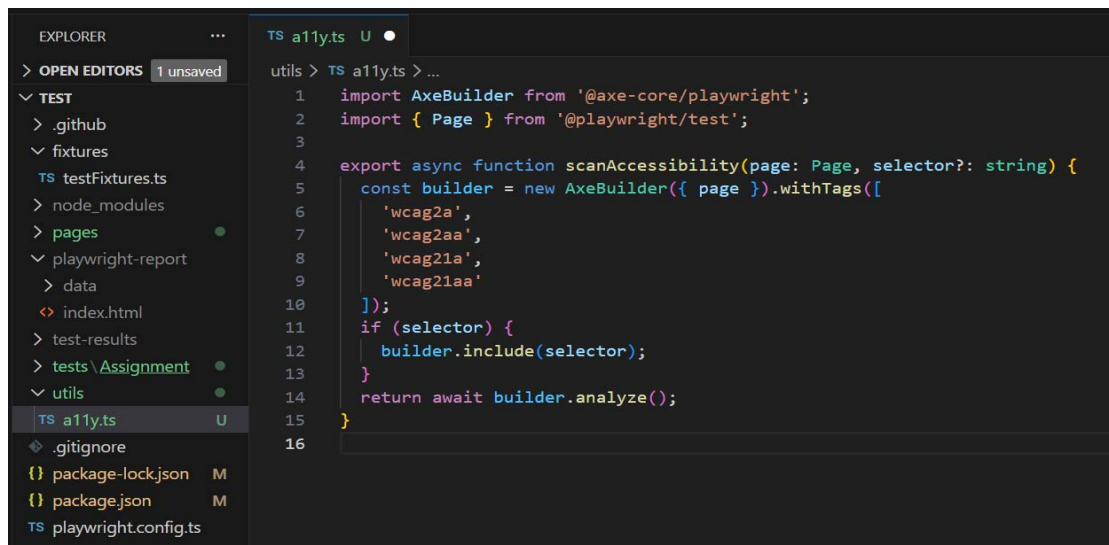


Reported By: Ping CHEN

Date:07/04/2026

2. Axe-core page scan tests

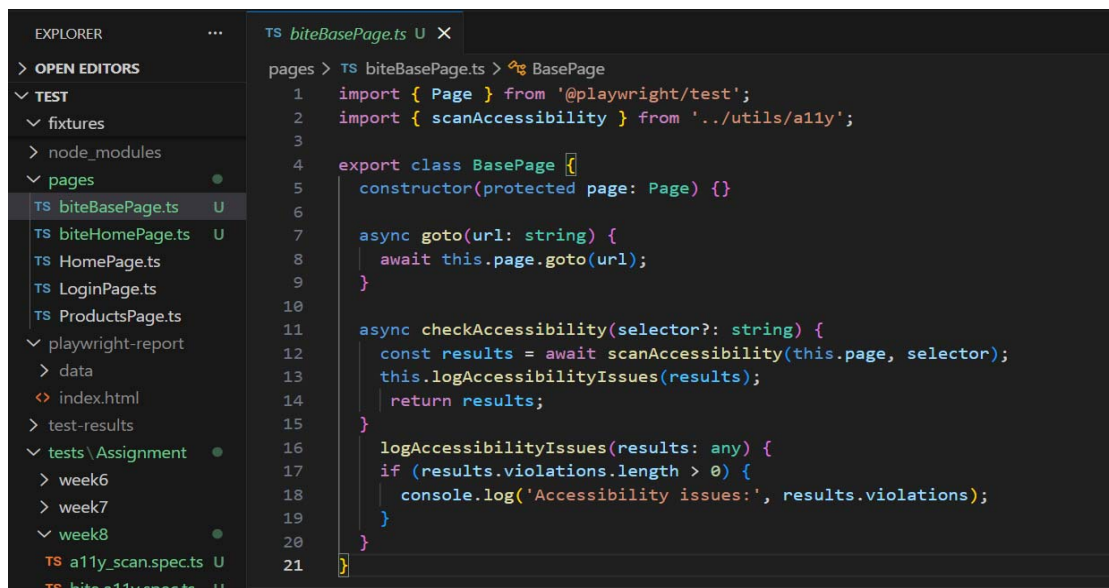
Utils:



```
EXPLORER
...
> OPEN EDITORS 1 unsaved
TEST
> .github
> fixtures
TS testFixtures.ts
> node_modules
> pages
playwright-report
> data
  index.html
> test-results
> tests\Assignment
utils
  TS a11y.ts U
  .gitignore
  package-lock.json M
  package.json M
  playwright.config.ts

TS a11y.ts U
utils > TS a11y.ts > ...
1  import AxeBuilder from '@axe-core/playwright';
2  import { Page } from '@playwright/test';
3
4  export async function scanAccessibility(page: Page, selector?: string) {
5      const builder = new AxeBuilder({ page }).withTags([
6          'wcag2a',
7          'wcag2aa',
8          'wcag21a',
9          'wcag21aa'
10     ]);
11     if (selector) {
12         builder.include(selector);
13     }
14     return await builder.analyze();
15 }
16
```

BasePage:



```
EXPLORER
...
> OPEN EDITORS
TEST
> fixtures
> node_modules
pages
  TS biteBasePage.ts U
  TS biteHomePage.ts U
  TS HomePage.ts
  TS LoginPage.ts
  TS ProductsPage.ts
playwright-report
> data
  index.html
> test-results
> tests\Assignment
  week6
  week7
  week8
  TS a11y_scan.spec.ts U
  TS bite a11y spec.ts U

TS biteBasePage.ts U X
pages > TS biteBasePage.ts > BasePage
1  import { Page } from '@playwright/test';
2  import { scanAccessibility } from '../utils/a11y';
3
4  export class BasePage {
5      constructor(protected page: Page) {}
6
7      async goto(url: string) {
8          await this.page.goto(url);
9      }
10
11     async checkAccessibility(selector?: string) {
12         const results = await scanAccessibility(this.page, selector);
13         this.logAccessibilityIssues(results);
14         return results;
15     }
16     logAccessibilityIssues(results: any) {
17         if (results.violations.length > 0) {
18             console.log('Accessibility issues:', results.violations);
19         }
20     }
21 }
```


HomePage:

EXPLORER
OPEN EDITORS 1 unsaved
TEST
fixtures
node_modules
pages
biteBasePage.ts U
biteHomePage.ts U
HomePage.ts
LoginPage.ts
ProductsPage.ts
playwright-report
data
index.html
test-results
tests Assignment
week6
week7
week8

TS biteHomePage.ts U
pages > TS biteHomePage.ts > HomePage > checkProductCardAccessibility
1 import { BasePage } from './biteBasePage';
2
3 export class HomePage extends BasePage {
4 async open() {
5 await this.goto('https://bitetoothpastebits.com/');
6 }
7 async checkHomeAccessibility() {
8 return await this.checkAccessibility();
9 }
10 async checkNavbarAccessibility() {
11 return await this.checkAccessibility('#desktopNav');
12 }
13 async checkProductCardAccessibility() {
14 return await this.checkAccessibility('[class*="product"]');
15 }
16 async checkFooterAccessibility() {
17 return await this.checkAccessibility('#footer');
18 }
19 }

Tests:

EXPLORER
OPEN EDITORS
TEST
fixtures
node_modules
pages
biteBasePage.ts U
biteHomePage.ts U
HomePage.ts
LoginPage.ts
ProductsPage.ts
playwright-report
data
index.html
test-results
tests Assignment
week6
week7
week8
a11y_scan.spects U
a11y.spects U
utils
a11y.ts U
gitignore
package-lock.json M
narkann.json M
OUTLINE
TIMELINE

TS bite.a11y.spects U X
tests > Assignment > week8 > TS bite.a11y.spects > test.describe('Accessibility Tests') callback > test('Home page should have no violations') callback
4 test.describe('Accessibility Tests', () => {
5 test('Home page should have no violations', async ({ page }) => {
6 // ...
7 });
8
9 test('Navbar should have no violations', async ({ page }) => {
10 const home = new HomePage(page);
11
12 await home.open();
13
14 const results = await home.checkNavbarAccessibility();
15
16 expect(results.violations).toEqual([]); - 4ms
17 });
18
19 test('Product Card should have no violations', async ({ page }) => {
20 const home = new HomePage(page);
21
22 await home.open();
23
24 const results = await home.checkProductCardAccessibility();
25
26 expect(results.violations).toEqual([]); - 4ms
27 });
28
29 test('Footer should have no violations', async ({ page }) => {
30 const home = new HomePage(page);
31
32 await home.open();
33
34 const results = await home.checkFooterAccessibility();
35
36 expect(results.violations).toEqual([]); - 4ms
37 });
38
39 });
40 }

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
TEST RESULTS
PORTS
PLAYWRIGHT

Playwright Test for VSCode
3 failed
[chromium] > tests(Assignment\week8\bite.a11y.spec.ts:6:7 > Accessibility Tests > Home page should have no violations Error: expect(received).toHaveLength(0) but got: 1
[chromium] > tests(Assignment\week8\bite.a11y.spec.ts:25:7 > Accessibility Test s > Product Card should have no violations Error: expect(received).toHaveLength(0) but got: 1
[chromium] > tests(Assignment\week8\bite.a11y.spec.ts:34:7 > Accessibility Test s > Footer should have no violations Error: expect(received).toHaveLength(0) but got: 1
1 passed (29.0s)

Search tests

All 4 ✓ Passed 1 ✗ Failed 3 Flaky 0 Skipped 0

Project: chromium 4/9/2026, 10:34:59 PM Total time: 29.0s

Assignment/week8/bite.a11y.spec.ts	
✗ Accessibility Tests > Home page should have no violations chromium	23.5s
Assignment/week8/bite.a11y.spec.ts:6	
✗ Accessibility Tests > Product Card should have no violations chromium	21.8s
Assignment/week8/bite.a11y.spec.ts:25	
✗ Accessibility Tests > Footer should have no violations chromium	20.9s
Assignment/week8/bite.a11y.spec.ts:34	
✓ Accessibility Tests > Navbar should have no violations chromium	19.0s
Assignment/week8/bite.a11y.spec.ts:16	

3. Report analysis with conclusions

My Axe-core page scan tests covered four accessibility areas: the homepage, navigation bar, product card section, and the footer. The results are as follows:

Name	Homepage test	Navbar test	Product card test	Footer test.
Errors	1889	0	255	766

Impact that is critical:

Certain ARIA roles must contain particular children;

Element has children which are not allowed: li[tabindex]".

Impact that is serious:

 elements must be contained in a or ";

List item parent element has a role that is not role=\"list\"";

Links must have discernible text;

Element does not have text that is visible to screen readers;

Aria-label attribute does not exist or is empty.

For WAVE, it shows one empty button error, eight low contrast errors, and 32 alerts, for example, redundant alternative text and link, long alternative text, skipped heading level, suspicious link text, element with no script, very small text.

For lighthouse, it found the following issues: Elements with an ARIA [role] that require children to contain a specific [role] are missing some or all of those required children. [role] values are not valid. Image elements do not have [alt] attributes. Links do not have a discernible name. List items () are not contained within , or <menu> parent elements.

For manual testing, I used the NVDA screen reader and keyboard navigation and identified five accessibility issues. For example, I found insufficient color contrast, and some elements such as list sequence, images, and rating stars were not properly announced by the screen reader. However, collapsed accordions were announced by NVDA, so some hidden components are accessible.

Manual testing provides a human perspective, allowing us to experience the website in a similar way to real users with assistive technologies. It is limited in scale, but it can sometimes expose usability issues that automated tools may miss. In contrast, automated testing is useful for quick and scalable detection of common accessibility problems. Some issues found during manual testing overlap with those found by automated tools, while others are unique to manual exploration. Thus, combining both approaches provide a much more complete and reliable accessibility evaluation.